DECA Lab Logbook — Spring Term

Lab 1 | Task 1: Write Address Control

Task 1: Write Address Control

Objectives

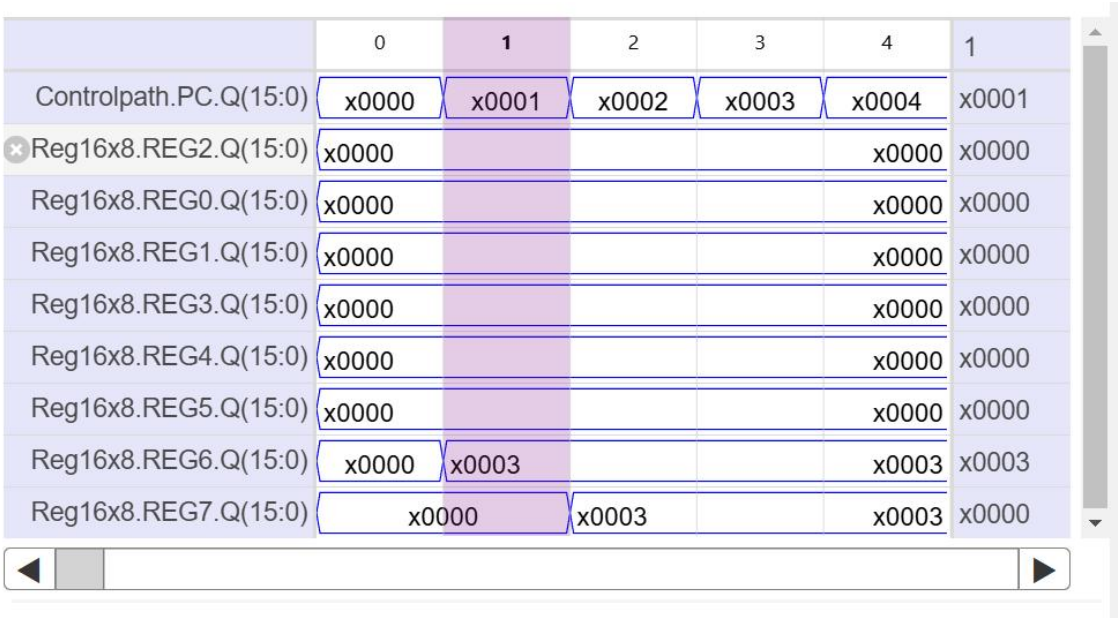
- How to use the waveform simulator to analyse how data moves in hardware
- How fields within the instruction word control reading from and writing to registers
- Which pieces of hardware implement this control in the EEP1 datapath

1. Write address control

```
MOV R6, #3
MOV R7, R6
```

1: For these MOV instructions, do the registers change in the same cycle as the instruction that changes their values, or in the following cycle?

Registers update in the following cycle. The waveform viewer shows that register values change one clock edge after the instruction that writes them is executed — MOV R6, #3 executes in cycle 0 and R6 reflects the new value in cycle 1.



	0	1	2	3	4	1
Controlpath.PC.Q(15:0)	x0000	x0001	x0002	x0003	x0004	x0001
Reg16x8.REG2.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG0.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG1.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG3.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG4.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG5.Q(15:0)	x0000				x0000	x0000
Reg16x8.REG6.Q(15:0)	x0000	x0003			x0003	x0003
Reg16x8.REG7.Q(15:0)	x0000		x0003		x0003	x0000

2: Using the EEP1 documentation, confirm that for register-register MOV instructions (where INS(8) = 1) the field a (written as Ra in assembly) determines which register is written in the register file. Then, using the ALU Instruction Encoding sheet, state exactly which bits of the 16-bit instruction word form field a.

For register-register MOV (INS(8) = 1), field a specifies the destination register. From the ALU Instruction Encoding sheet:

- Write register: Ra (field a)
- Field a = bits[11:9] of the 16-bit instruction word

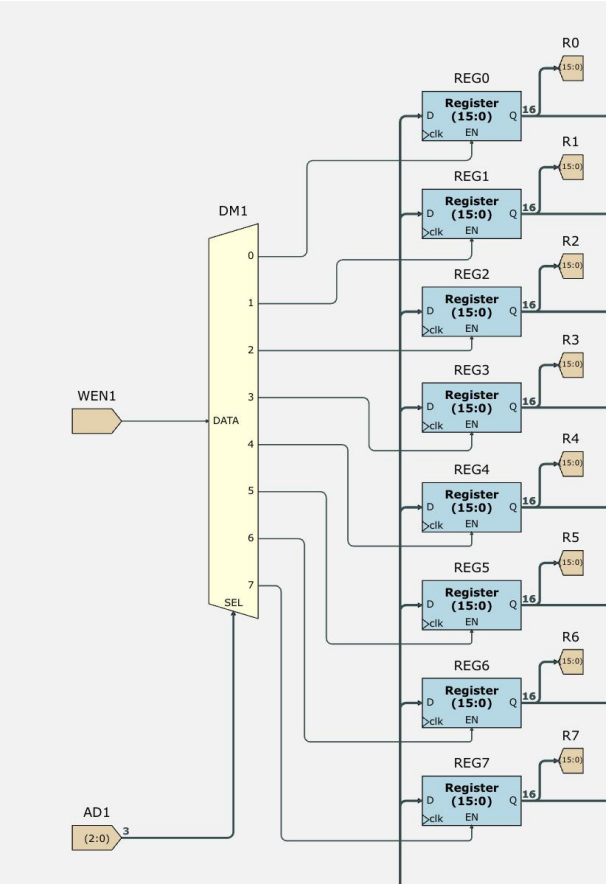
The graphs are in the next page.

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1	Write FlagC[1]	Write register
0	MOV	Rb	Imm	No	Ra

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

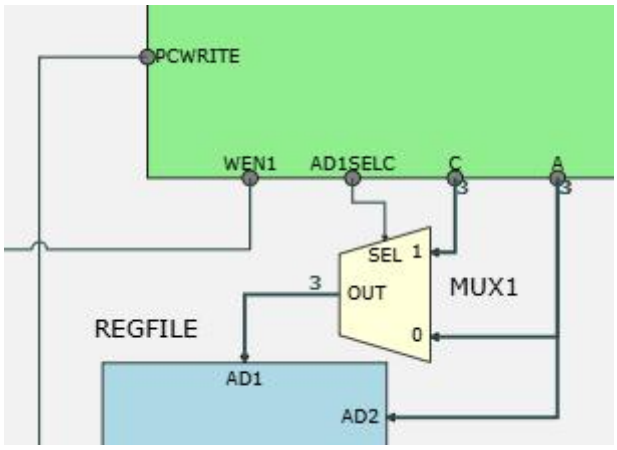
3: Open the regfile sheet. Examine the names and widths of its input and output ports. From the port names (and the hardware they drive), deduce which input port controls the address of the register that is written. You should find that this port is AD1.

The write address is controlled by AD1. AD1 connects to the select input of the 1-to-8 DEMUX on the write side of the register file. Each DEMUX output drives one register's enable line, so the 3-bit value on AD1 determines which register latches the incoming data on the rising clock edge



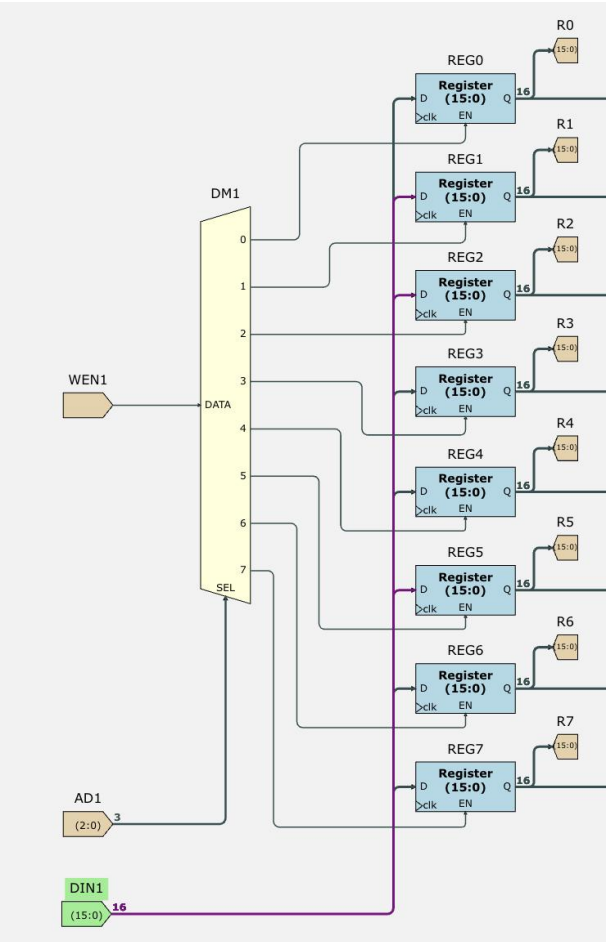
4: Trace the wire connected to AD1 back into the dpdecode block. The purpose of dpdecode is to interpret the 16-bit instruction word each cycle and to generate the appropriate control signals for the CPU. By I nspecting the hardware, determine which bits of the 16-bit instruction word drive the signal connected to AD1, and compare this with your earlier answer about which bits encode field a. (For this investigation you may assume that AD1SELC = 0.)

With AD1SELC = 0, the MUX inside



dpdecode selects output port A, which connects to AD1. Port A is wired to bits[11:9] of the instruction word — identical to field a. The write address is therefore decoded directly from the instruction.

[9:11] is connected to AD1



5: Return to the register file and trace what hardware the AD1 input connects to internally. Through which port does the data being written enter the register file? Explain how this hardware is arranged so that the value on AD1 selects which register the incoming data is written to.

Write data enters through port DIN1. Internally, AD1 (3 bits) addresses the DEMUX, which asserts the enable signal of exactly one of the eight registers. On the rising clock edge only that register latches the DIN1 value; the other seven are unaffected.

6: Using the waveform viewer, track the value of AD1 in each cycle. Does it correspond to the register that is written in the next cycle, as you expect?

Yes. AD1 is driven combinationally in the same cycle as instruction fetch, so the value of AD1 in cycle N identifies the register written in cycle N+1. For example, AD1 = 6 in cycle 0 corresponds to R6 being updated in cycle 1.

	0	1	0
Datapath.REGFILE.Ad1(2:0)	6 6	7 7	6
Controlpath.PC.Q(15:0)	x0000	x0001	x0000
Reg16x8.REG2.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG0.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG1.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG3.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG4.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG5.Q(15:0)	x0000	x0000	x0000
Reg16x8.REG6.Q(15:0)	x0000	x0003	x0000
Reg16x8.REG7.Q(15:0)	x0000	x0000	x0000

2. Read address control

1: Using ALU Instructions, confirm that for a MOV instruction with INS(8) = 0 the field b (written as Rb) selects the register from which data is read in the register file. Using the ALU Instruction Encoding sheet, identify exactly which bits of the instruction form field b.

For MOV with INS(8) = 0, field b (Rb) selects the source register.

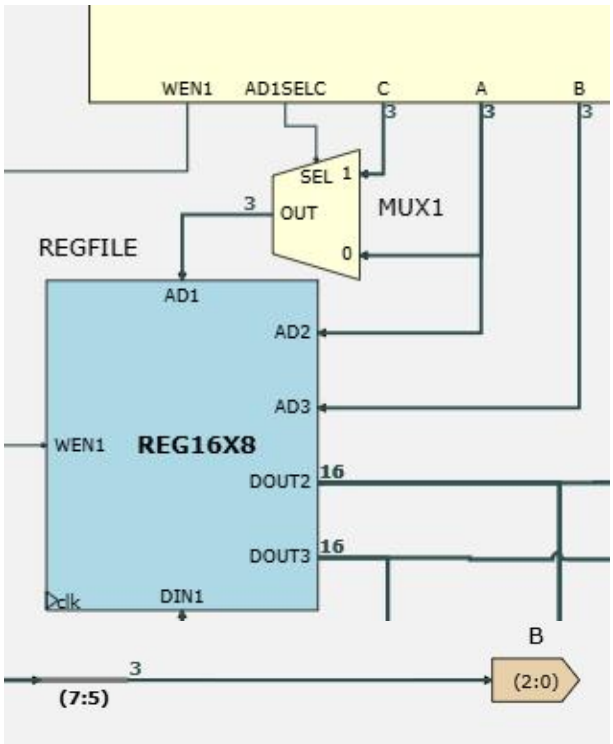
- Read register: Rb (field b)
- Field b = bits[7:5] of the 16-bit instruction word

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1	Write FlagC[1]	Write register
0	MOV	Rb	Imm	No	Ra

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

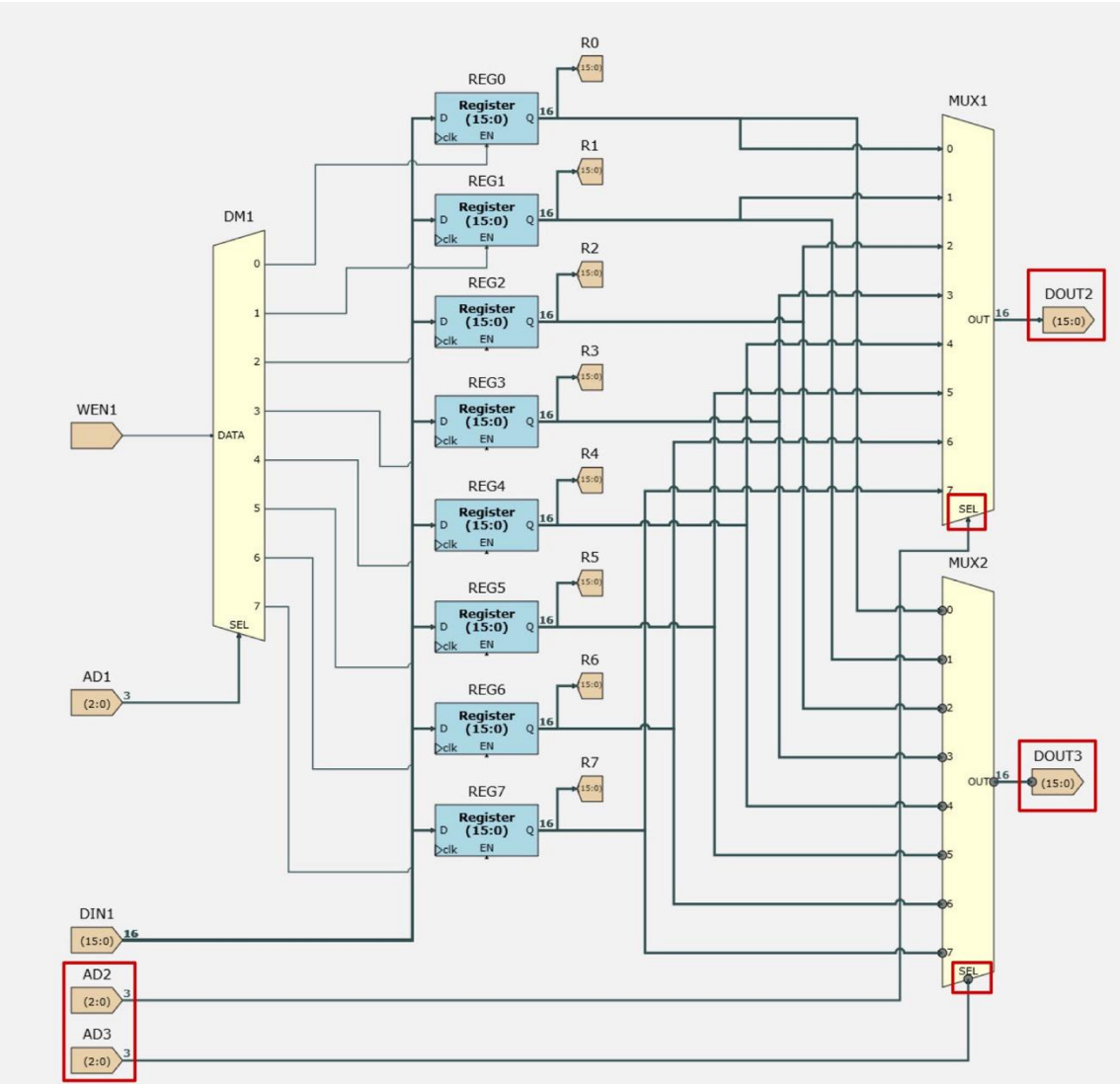
2: Open dpdecode and trace the wire that carries the bits of field b. Note which port of the register file it connects to. This port is the one that controls the read address for these register-to-register MOV instructions.

Port B of dpdecode carries bits[7:5] and connects to AD3 of the register file. AD3 is the read address for the second data output.

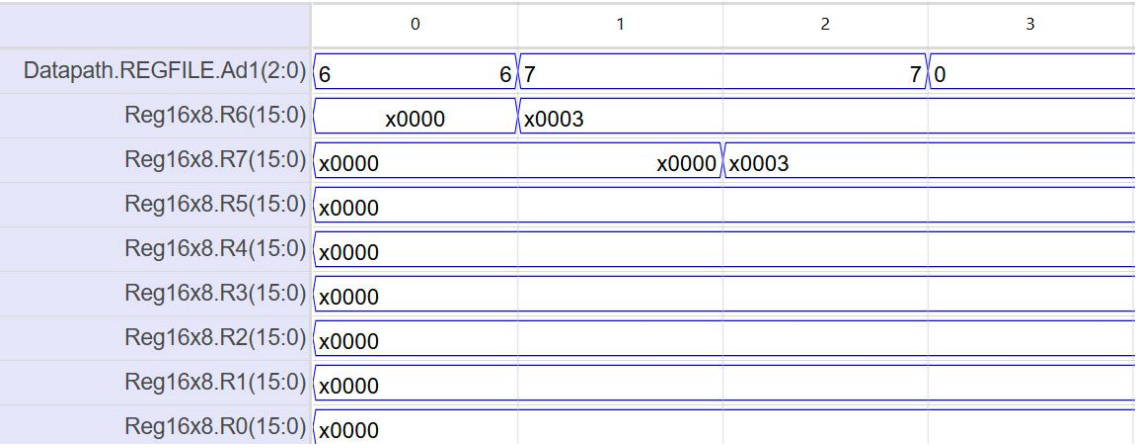


3: Open the register file and follow the hardware connected to that read-address port. Explain how the hardware uses the input signal to select which register's data is placed on the register file's output bus, and state which of the two data output ports this data leaves through.

AD2 and AD3 each index a dedicated 8-to-1 MUX whose inputs are the Q outputs of all eight registers. AD2 selects one register value and delivers it to DOUT2 (→ ALU port RA); AD3 selects another and delivers it to DOUT3 (→ ALU port RB). For a register-to-register MOV, the source register value exits through DOUT3.



Extension

In a register-register MOV, what happens if a = b (for example MOV R7, R7)? Does this cause any problems?

No problems arise. The read ports output the current value of R7 combinationally. The write port then stores that same value back into R7 on the next clock edge, leaving the register content unchanged. The waveform shows AD1 = 7 for two consecutive cycles, with R7 remaining stable throughout.